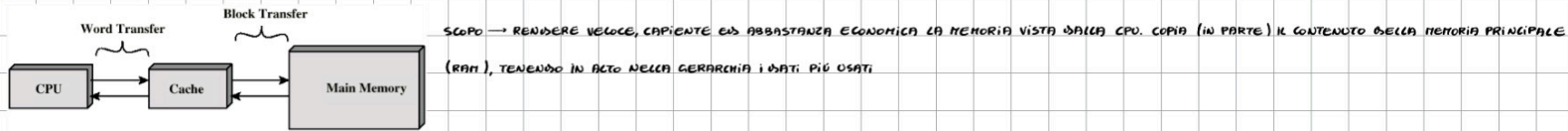


MEMORIA CACHE

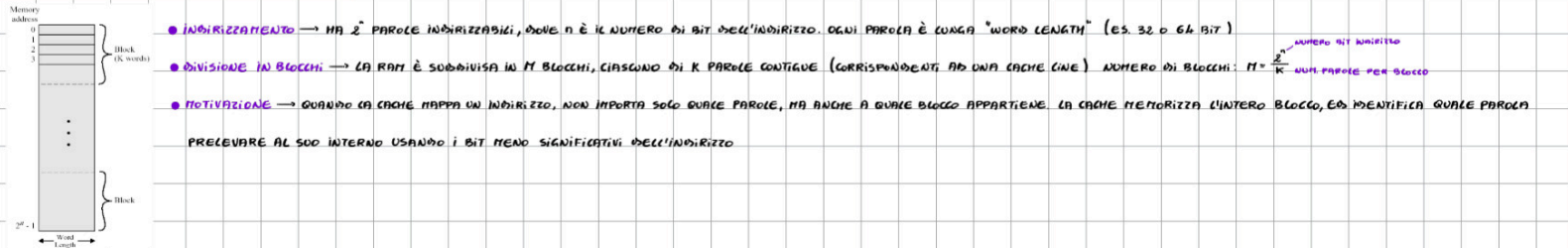


● POSIZIONE FISICA → PUÒ TROVARSI DENTRO IL CHIP CPU (CACHE L1/L2) O IN UN MODULO ESTERNO SUL BUS (CACHE L3)

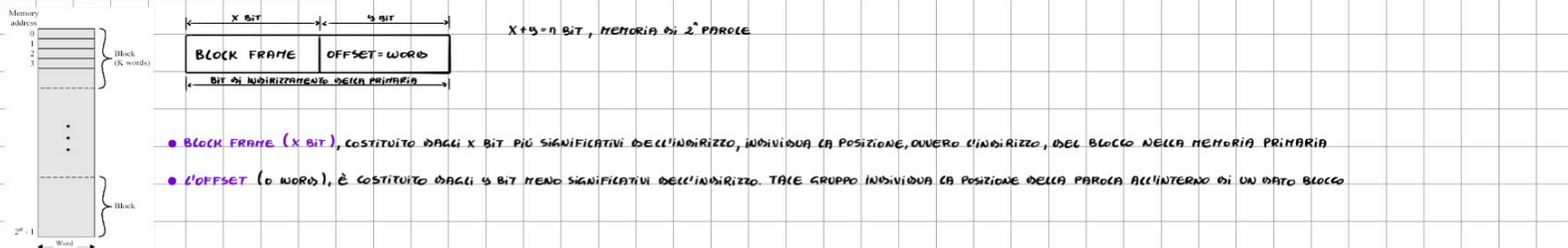
● MODALITÀ DI TRASFERIMENTO:

- WORD TRANSFER (TRA CPU E CACHE): LA CPU LEGGE/SCRIVE UNA SINGOLA PAROLA ALLA VOLTA, TIPICAMENTE (ARCA 32 O 64 BIT)
- BLOCK TRANSFER (TRA CACHE E RAM): QUANDO LA CACHE NON CONTIENE LA PAROLA RICHIESTA (CACHE MISS), LEGGE UN INTERO BLOCCO DI K PAROLE CONTIGUE DALLA RAM, SFRUTTANDO LA LOCALITÀ SPAZIALE

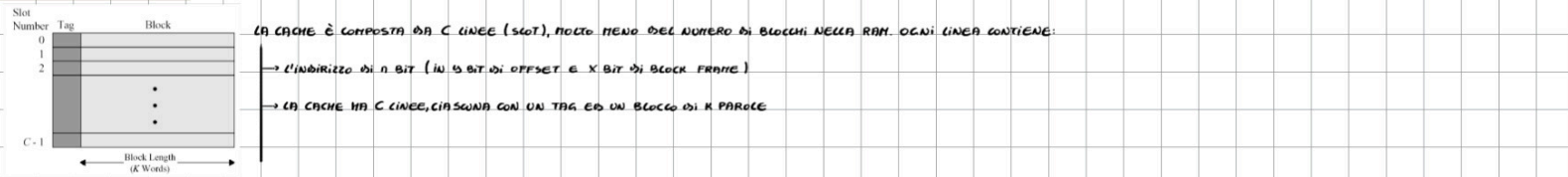
STRUTTURA LOGICA DELLA MEMORIA PRIMARIA



INDIRIZZAMENTO DELLA MEMORIA PRIMARIA



STRUTTURA DELLA CACHE

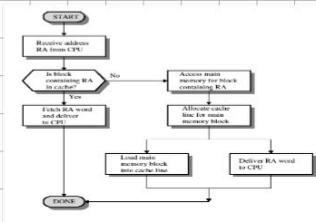


- Cache piccola e veloce. In un certo intervallo di tempo, può contenere solo un piccolo sottoinsieme dei blocchi di primaria
- Poiché ci sono più blocchi di primaria che linee di cache, un'incerta linea non può essere assegnata in modo univoco ad un unico blocco di primaria
- Il "tag" identifica il blocco che in un certo istante è presente in una linea di cache. Vediamo nelle pagine seguenti come è fatto il "tag" (etichetta)
- Voglio fare in modo che nella maggior parte degli accessi del processore alla memoria la parola cercata sia già in cache
- Parleremo di cache "hit" e cache "miss"

FUNZIONAMENTO DI PRINCIPIO DELLA CACHE

La CPU richiede una parola di memoria primaria inviando l'indirizzo che la parola ha in primaria

- Si controlla se la parola è presente in cache.
- Se è presente, si prende la parola dalla cache
- Ciò avviene molto velocemente, perché la cache è una memoria con basso tempo di accesso.
- Se non è presente, si copia dalla memoria principale in cache il blocco a cui appartiene la parola
- Si copia un blocco, e non una sola parola, per sfruttare il Principio di Località
- Poi si trasferisce la parola dalla cache alla CPU
- La cache include un campo, detto "tag", per identificare quali blocchi della memoria principale sono presenti nelle sue "linee" in un certo istante.

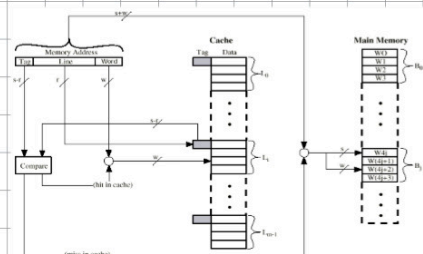


PROGETTAZIONE DELLA CACHE

Cache Size DIMENSIONE CACHE	Write Policy	VI SONO DIVERSE IMPLEMENTAZIONI, DISTINGUIBILI TRAMITE I SEGUENTI ELEMENTI CHIAVE
Mapping Function FUNZIONE MAPPAZIONE	Write through	
Direct	Write back	
Associative	Write once	
Set Associative		
Replacement Algorithm	Line Size	
Least recently used (LRU)	Number of caches	
First in first out (FIFO)	Single or two level	
Least frequently used (LFU)	Unified or split	
Random		

GRANDEZZA DELLA CACHE

- LA CACHE DEVE BILANCIARE COSTO E PRESTAZIONI:
- **COSTO** → UNA CACHE PICCOLA COSTA MENO E HA CATENA INFERIORE (TEMPO DI ACCESSO)
 - **VELOCITÀ E HIT RATE** → UNA CACHE GRANDE CONTIENE PIÙ BLOCCHI DI RAM E PORTA AD UN HIT RATE PIÙ ALTO (MENO MISS, QUINDI MENO ACCESSI LENTI ALLA RAM). TUTTAVIA, UNA CACHE PIÙ GRANDE RICHIEDE PIÙ TEMPO PER CONFRONTARE I TAG O PER GESTIRE I SET
- Trade-off: tipicamente le cache L1 sono piccole (da decine a qualche centinaio di KB) per essere rapidissime, mentre cache L2/L3 salgono a qualche MB per aumentare l'hit rate complessivo. Non esiste "taglia unica": dipende dal tipo di codice e dal principio di località che sfrutta.



METODO DIRETTO: MAPPATURA FRA CACHE E MEMORIA PRIMARIA

Linea di Cache	Indirizzo di inizio del blocco di memoria primaria allocato nella linea di cache
0	000000, 010000, ..., FF0000
1	000004, 010004, ..., FF0004
.	.
m-1	00FFFC, 01FFFC, ..., FFFFFC

PRENDIAMO I DUE PARAMETRI GIÀ VISTI

$m = 2^k = 16 \cdot 32k$ LINEE DI CACHE

BLOCCHI DI RAM DA 4 BYTE → OGNI BLOCCO HA INIZIO E FINE OCCUPA 4 BYTE CONSECUTIVI

$i = j \bmod m \rightarrow i = \frac{j}{m}$

SE I È L'INDIRIZZO DELLA LINEA DELLA CACHE, È L'INDIRIZZO DEL BLOCCO DI MEMORIA E M IL NUMERO DI LINEE DELLA CACHE. L'ALLOCAZIONE DI UN BLOCCO DI MEMORIA PRIMARIA IN CACHE SEGUE LA LEGGE $i = j \bmod m$ NEL NOSTRO ESEMPIO CON $m = 16 \cdot k = 2^4$ $i = j \bmod 2^4$

• LINEA CACHE 0 → DOPO BLOCCO DI PRIMARIA CON L'INDIRIZZO ESADECIMALE DI INIZIO 000000 ABBIAMO BLOCCO CON INDIRIZZO DI INIZIO ESADECIMALE 010000 CHE CORRISPONDE AL BLOCCO $m = 16 \cdot k = 2^4 = 00 \cdot 0100 \cdot 000$

IL BLOCCO $m = 16 \cdot k = 2^4$ INIZIA ALL'INIZIO 010000 PERCHÉ I BLOCCHI CONTENGONO 4 BYTE = 2^2

CACHE LINEA $m = 2^4 = 16 \cdot 32k$

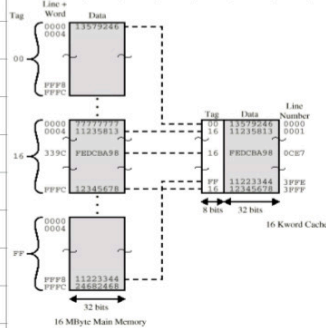
BLOCK SIZE = 4 BYTE (QUINDI OGNI BLOCCO HA $W = 2$ BIT DI OFFSET)

OGNI BLOCCO SI INIZIA IN RAM ALL'INDIRIZZO $START = j \cdot k$ (BYTE) $i = j \bmod 2^4$

6 CIFRE
TUTTI I BLOCCHI j CON $j \bmod 2^4 = 0 \rightarrow$ LINEA 0
QUESTI j SONO $\rightarrow j = 2^4 \cdot 0, 2^4 \cdot 2, 2^4 \cdot 3, \dots$
 $m = 2^4 = 00 \cdot 0100 \cdot 000 \cdot 2^2 = 0 \cdot 0001 \cdot 0000$

Linea di Cache	Indirizzo di inizio del blocco di memoria primaria allocato nella linea di cache
0	000000, 010000, ..., FF0000
1	000004, 010004, ..., FF0004
.	.
m-1	00FFFC, 01FFFC, ..., FFFFFC

$j = 2^4 \cdot 0 \quad 0 \cdot 4 = 0 \quad 0X000000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 1 \quad 2^4 \cdot 4 = 65536 \quad 0X010000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 2 \quad 32768 \cdot 4 = 131072 \quad 0X020000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 3 \quad 48192 \cdot 4 = 192640 \quad 0X030000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 4 \quad 64640 \cdot 4 = 258176 \quad 0X040000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 5 \quad 81088 \cdot 4 = 323712 \quad 0X050000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 6 \quad 97536 \cdot 4 = 389248 \quad 0X060000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 7 \quad 113984 \cdot 4 = 454784 \quad 0X070000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 8 \quad 130432 \cdot 4 = 520320 \quad 0X080000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 9 \quad 146880 \cdot 4 = 585856 \quad 0X090000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 10 \quad 163328 \cdot 4 = 651392 \quad 0X0A0000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 11 \quad 179776 \cdot 4 = 716928 \quad 0X0B0000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 12 \quad 196224 \cdot 4 = 782464 \quad 0X0C0000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 13 \quad 212672 \cdot 4 = 848000 \quad 0X0D0000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 14 \quad 229120 \cdot 4 = 913536 \quad 0X0E0000$ (IN ESADECIMALE)
 $j = 2^4 \cdot 15 \quad 245568 \cdot 4 = 979072 \quad 0X0F0000$ (IN ESADECIMALE)



- Vantaggi
 - Semplice
 - Poco costoso da implementare

- Principale svantaggio
 - Locazione fissa per un dato blocco.
- Se vengono richiesti alternativamente due blocchi mappati nella stessa linea di cache, questi vengono continuamente scambiati e riducono notevolmente l' "hit ratio".

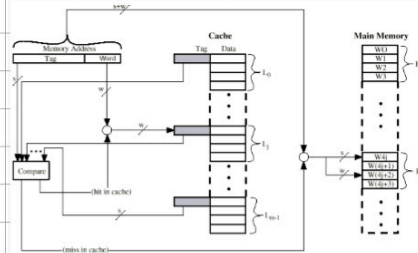
METODO ASSOCIATIVO

UN BLOCCO PUÒ OCCUPARE QUALSIASI LINEA DI CACHE. LA CACHE DEVE CONTROLLARE TUTTI I TAG IN PARALLELO (O CON LOGICA CAN)

PRO → MASSIMA FLESSIBILITÀ, MENO MISS PER CONFLITTO

CONTRO → COSTOSA E LENTA (MOLTI CONFRONTI DI TAG SIMULTANEI)

- Un blocco della memoria principale può essere caricato all'interno di qualsiasi linea di cache
- L'indirizzo di memoria principale è interpretato come "tag" e "word" (non c'è cache index)
- Il tag identifica un blocco di memoria primaria
- Ciascun campo tag nella cache viene esaminato per un confronto al fine di individuare il blocco di memoria cercato
- La ricerca in cache diventa più complessa, dovendo effettuare dei confronti multipli



TAG 22 BIT	WORD 2 BIT
------------	------------

- IL TAG DA 22 BIT DEVE ESSERE MEMORIZZATO IN CACHE PER OGNI LINEA DI DATI (DA 4 BYTE)
- SI CONFRONTANO IL CAMPO DEL TAG DI TUTTE LE LINEE CON IL TAG IN ENTRATA ("INDIRIZZO") NELLA CACHE
- I 2 BIT MENO SIGNIFICATIVI DELL'INDIRIZZO IDENTIFICANO QUALE PAROLA DA 4 BYTE È RICHIESTA NEL BLOCCO DI DATI, QUINDI
- PER OTTENERE IL TAG SI FA SCORRERE L'INDIRIZZO VERSO DESTRA DI 2 POSIZIONI, CIÒ È DEL NUMERO DI BIT NECESSARI PER INDIRIZZARE UNA PAROLA; QUINDI

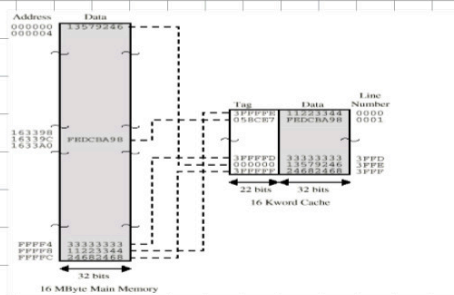
SI DIVIDE PER 2^2 IN GENERALE:

$TAG = \frac{INT(INDIRIZZO \cdot COMPLESSIVO)}{2^W}$
 $WORD = \frac{MOD(INDIRIZZO \cdot COMPLESSIVO)}{2^W}$

- Principale vantaggio
 - flessibilità nell'allocazione dei blocchi in cache
- Principale svantaggio
 - complessità del «circuit» richiesto per esaminare i "tag" di tutte le linee di cache in parallelo

PAROLA (WORD) = 4 BYTE PER BLOCCO → SERVE UN OFFSET DI $W = 2$ BIT PER SELEZIONARE IL BYTE DENTRO IL BLOCCO

TAG = TUTTI I RESTANTI BIT DELL'INDIRIZZO RAM. SE L'INDIRIZZO È COMPLESSIVAMENTE N BIT, QUI $TAG = N - W$ (ES. $N = 24$ $TAG = 24 - 2 = 22$ BIT)



ESEMPIO

0001	0110	0011	0011	1001	1100	INDIRIZZO
1	6	3	3	9	C	
01	0101	1000	1000	1110	0111	TAG
0	5	8	C	E	7	

METODO ASSOCIATIVO SU INSIEMI

• ORGANIZZAZIONE "A SET"

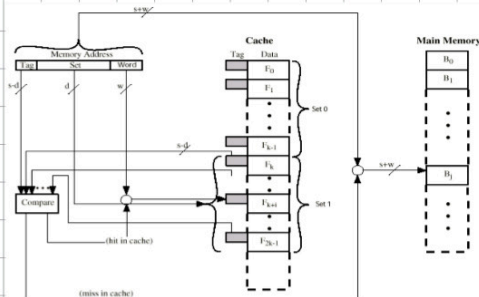
- LA CACHE È DIVISA IN V "INSIEMI" (SET)
 - OGNI SET CONTIENE K LINEE; PER QUESTO SI PARLA DI K -WORD SET-ASSOCIATIVE
 - UN BLOCCO DI RAM MAPPA SEMPRE IN UN SOLO SET, MA PUÒ OCCUPARE UNA QUALSIASI DELLE K LINEE DI QUEL SET (COME IN FULLY-ASSOCIATIVE)
- TIPICAMENTE $K = 2$ O 4 PER UN BUON COMPROMESSO TRA COMPLESSITÀ HARDWARE E RIDUZIONE DEI CONFLITTI

• FORMATO DELL'INDIRIZZO

SE L'INDIRIZZO RAM È N BIT, E LA CACHE È ORGANIZZATA IN SET-ASSOCIATIVE CON BLOCCHI DI 2^W PAROLE E $V = 2^J$ SET, ALLORA:

- WORD (W BIT) → IDENTIFICA LA PAROLA/BYTE NEL BLOCCO
- SET INDEX (J BIT) → DETERMINA QUALE DEI 2^J SET CONSULTARE
- TAG ($S = R$ BIT)

TAG S-R	SET R	WORD W
9	13	2

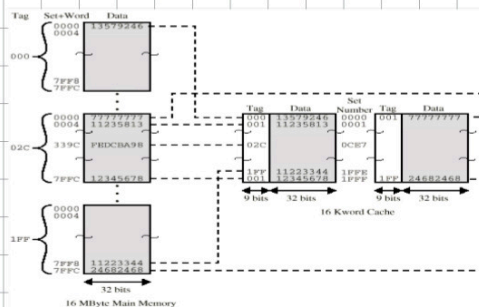


ESEMPIO METODO ASSOCIATIVO SU INSIEMI DA 2 LINEE: FORMATO DELL'INDIRIZZO

TAG S-R	SET R	WORD W
9	13	2

INDIRIZZO	TAG	DATO	INDIRIZZO DEL SET
1FF 7FFC	1FF	12345678	1FFF
001 7FFC	001	11223344	1FFF

- INDIRIZZO SET + WORD = 7FFC, DA CUI SI PUÒ RICAVARE INDIRIZZO "SET" DIVIDENDO PER 4 (DIMENSIONE BLOCCHI CACHE), CHE CORRISPONDE AD UNO SPOSTAMENTO VERSO DESTRA DI 2 POSIZIONI
- PER CUI 0111 1111 1111 1100 DIVENTA 01 1111 1111 1111 = 1FFF
- NOTARE CHE I DUE BLOCCHI HANNO STESSO SET INDEX MA DIVERSO TAG



Metodo associativo su insiemi da 2 linee: relazioni utili (1)

Blocco s	Word w
22	2

- Per determinare l'indirizzo del blocco di memoria si deve traslare l'indirizzo complessivo di tante posizioni quanti sono i bit necessari per identificare una parola; ciò corrisponde a dividere per 2^w
 - indirizzo_blocco = Int(indirizzo_complessivo/2^w)
 - indirizzo_word = Mod(indirizzo_complessivo/2^w)

Metodo associativo su insiemi da 2 linee: relazioni utili (2)

Tag s-r	Insieme r
9	13

- Per determinare il tag si fa scorrere l'indirizzo a destra del numero di bit necessario per rappresentare gli insiemi, cioè si divide per 2^r
 - tag = Int(indirizzo_blocco/ 2^r)
 - indirizzo_insieme = Mod(indirizzo_blocco/ 2^r)

ALGORITMI DI RIMPIAZZAMENTO

QUANDO LA CACHE È PIENA ED ARRIVA UNA CACHE MISS, BISOGNA SORRISCRIVERE ("RIMPIAZZARE") UN BLOCCO GIÀ PRESENTE

DIRECT-MAPPED

POLITICA OBBLIGATA= OGNI BLOCCO RAM HA UNA SOLA LINEA DI DESTINAZIONE

AL MISS SI RIMPIAZZA IL BLOCCO CHE GIÀ OCCUPA QUELLA LINEA

ASSOCIATIVO / SET-ASSOCIATIVE

QUI INVECE PIÙ BLOCCHI POSSONO COMPETERE PER LE STESSSE LINEE (TUTTE LE LINEE NEL SET), QUINDI SERVE UNA POLITICA HARDWARE PER SCEGLIERE LA VITIMA

LRU -> LEAST RECENTLY USED

SI ELIMINA IL BLOCCO MENO RECENTEMENTE USATO

IMPLEMENTAZIONE SEMPLICE PER 2 LINEE: (OGNI LINEA HA UN BIT "RECENTLY USED")

- A "1" SE È STATA UTILIZZATA DOPO L'ALTRA LINEA DEL SET
- A "0" SE NON LO È STATA

AL MISS SI SCEGLIE LA LINEA CON BIT=0 E POI SI AZZERANO I BIT PER RIFLETTERE L'USO

PRO -> OTTIMO HIT RATE, SFRUTTA LA LOCALITÀ TEMPORALE

CONTRO -> COMPLESSITÀ CRESCE CON L'ASSOCIATIVITÀ

FIFO -> FIRST IN FIRST OUT

SI ELIMINA IL BLOCCO CHE È IN CACHE DA PIÙ TEMPO (PRIMO ARRIVATO, PRIMO USCITO)

SI IMPLEMENTA CON UNA CODA PER LINEA, OPPURE UN PUNTATORE CHE AVANZA CICLICAMENTE

PRO -> MOLTO SEMPLICE

CONTRO -> PIÙ "GREEDY" DEL LRU, TENDE A PEGGIORARE SE UN BLOCCO VECCHIO VIENE ANCORA USATO

LFD -> LEAST FREQUENTLY USED

SI RIMPIAZZA IL BLOCCO CON IL CONTEGGIO DI ACCESSI PIÙ BASSO

OGNI LINEA TIENE UN CONTATORE INCREMENTATO AD OGNI HIT

PRO -> PRESERVA I BLOCCHI "CALDI" CHE HANNO ALTO TASSO DI UTILIZZO

CONTRO -> RICHIEDE CONTATORI PER LINEA

Scelta tipica nei processori:

RANDOM

SI SCEGLIE CASUALMENTE UNA DELLE LINEE CANDIDATE

PRO -> HARDWARE MINIMALISTA, EVITA CONFLITTI

CONTRO -> NON SFRUTTA ALCUNA LOCALITÀ, HIT RATE MEDIO

2-way set-associative con LRU a bit: ottimo compromesso tra complessità e performance.

Nei set più grandi (4-way, 8-way) si adottano approssimazioni di LRU (pseudo-LRU) per ridurre il costo hardware.

LA TECNICA "WRITE THROUGH"

COSA FA -> OGNI VOLTA CHE LA CPU SCRIVE UN DATO IN CACHE, SCRIVE SIMULTANEAMENTE ANCHE NELLA MEMORIA PRINCIPALE (DRAM)

PRO

COERENZA SEMPLICE -> I DATI IN RAM SONO SEMPRE AGGIORNATI, NON SERVE TENERE TRACCIA DI QUALI LINEE SONO "SPORCHE"

IN SISTEMI MULTI-PROCESSOR, GLI ALTRI PROCESSORI POSSONO "SNOC" IL BUS DI MEMORIA ED INVALIDARE/AGGIORNARE LE PROPRIE CACHE IN TEMPO REALE

CONTRO

MOLTO TRAFFICO SUL BUS DI MEMORIA -> OGNI STORE GENERA UNA SCRITTURA IN DRAM, CREANDO UN POSSIBILE COLLO DI BOTTIGLIA

LATENZA DI SCRITTURA MAGGIORE PERCEPITA DAL CORE

TECNICA WRITE BACK

COSA FA -> LE SCRITTURE VENGONO FATE SOLO IN CACHE. OGNI LINEA HA UN DIRTY BIT -> 0 = PULITA (IDENTICA ALLA COPIA IN RAM) -> 1 = SPORCA (MODIFICATA, DIVERSA DA RAM)

QUANDO SI AGGIORNA LA RAM

SOLO AL RIMPIAZZO DI UNA LINEA "DIRTY": SE QUELLA LINEA DEVE ESSERE SOSTITUITA DA UN NUOVO BLOCCO, ALLORA LA CACHE INVIÀ IL BLOCCO MODIFICATO IN DRAM E RESETTA IL DIRTY BIT

RIDUCE IL NUMERO DI SCRITTURE IN MEMORIA PRINCIPALE (TIPICAMENTE SOLO IL 15% DELLE SCRITTURE SI PROPAGANO)

PRO

MENO TRAFFICO DI SCRITTURA SU DRAM -> MAGGIORE THROUGHPUT

LA CPU OTIENE LATENZE DI STORE PIÙ BASSE, PERCHÉ SCRIVE SOLO IN SRAM

Quando scegliere

Write-Through è più semplice e sicuro in ambienti SMP con bus condiviso, ma crea molto traffico di memoria.

Write-Back è più efficiente in single-processor o con meccanismi di coerenza avanzati (MESI, directory), e di gran lunga preferito nelle CPU moderne per ridurre il carico sulla DRAM.

CONTRO

SERVE GESTIRE IL DIRTY BIT PER OGNI LINEA

CASO DI POWER-FAILURE: I DATI MODIFICATI IN CACHE MA NON ANCORA SCRITTI IN RAM ANDRANNO PERSI

IN SISTEMI MULTI-PROCESSOR O DMA, BISOGNA GARANTIRE LA COERENZA: LE PERIFERICHE O LE ALTRE CPU DEVONO "PASSARE" ATTRAVERSO LA CACHE O INVALIDARSI A VICENDA

DIMENSIONE DI CASSAUA LINEA

PIÙ GRANDE SARA, MAGGIORMENTE SI SFROTA IL PRINCIPIO DI LOCALITÀ

PERCHÉ IMPORTA

LA CACHE TRASFERISCE E MEMORIZZA DATI IN BLOCCHI CHIAMATI "LINEE" (O "CACHE LINES"). LA DIMENSIONE DI CASSAUA LINEA (IN PAROLE O IN BYTE) GOVERNA QUANTO "GRASSO" È IL BLOCCO CHE VIENE PORTATO SU MISS.

→ LOCALITÀ SPAZIALE → LINEE PIÙ GRANDI CATTURANO PIÙ DATI CONTIGUI, RINDO SI SFROTA MEGLIO LOCALITÀ SPAZIALE SE IL PROGRAMMA LEGGE PER ESEMPIO ARRAI O FLOI DI DATI.

→ NUMERO DI LINEE DISPONIBILI → DATO UN BUDGET FISSO DI SRAM, LINEE PIÙ LUNGHE RIDUGONO IL NUMERO TOTALE DI LINEE (CAPACITÀ CACHE) CON IL RISCHIO DI AUMENTARE I CONFLITTI.

→ LATENCY DI FILL → TRASFERIRE LINEE TROPPO GROSSE RICHIEDE PIÙ TEMPO SU BUS VERSO LA RAM, AUMENTANDO LA LATENZA DI MISS.

• TRADE-OFF OTTIMALE → DIPENDE DAL CARICO DI LAVORO E DAL PROFILO D'ACCESSO, MA TYPICAMENTE SI USANO DA 2 A 8 PAROLE (8-64 BYTE SU CPU MODERNE).

LIVELLI DI CACHE

FINO A 3 LIVELLI VENGONO COMUNEMENTE IMPLEMENTATI NEI PROCESSORI MODERNI:

LIVELLO	NOME	CAPACITÀ
L1	LEVEL 1	10-100 KB
L2	LEVEL 2	512 KB-2 MB
L3	LEVEL 3	10 MB

CACHE I/O SEPARATE

L1 (ISTRUZIONI) E L1 (DATI) PER ELIMINARE CONFLITTI FRA FETCH E STORE.

GERARCHIA

→ LA CPU CERCA PRIMA IN L1 (SE HIT, ULTRA-VELOCE)

→ SE MISS → L2 (PIÙ GRANDE, UN PO' PIÙ LENTA)

→ ANCORA MISS → L3 (ANCORA PIÙ GRANDE E LENTA)

→ SE MISS SU L3 → RAM PRINCIPALE (LATENCY MASSIMA)

Motivazione

Allontanando un po' di capacità extra in livelli successivi, si ottiene sia il basso costo per bit che una latenza complessiva molto inferiore a cercare sempre in RAM.

In sintesi

Cache line size: equilibrio tra sfruttamento della località spaziale (linee grandi) e riduzione dei conflict/completi di fill (linee più piccole).

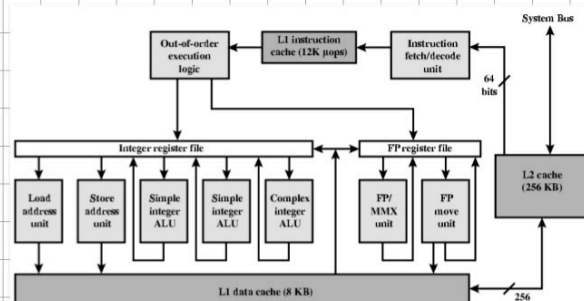
Cache levels: catena di SRAM di capacità crescente e latenza crescente, progettata per mantenere frequenti accessi all'interno dei livelli superiori più veloci, rimandando solo i rari miss ai livelli inferiori più lenti.

Esempi "storici" di uso cache in CPU

- Intel 80386 (1985) – Non supportava la cache interna.
- 80486 (1989) – Possedeva una cache interna da 8KB, in cui ogni linea poteva contenere un blocco da 16 byte ed era organizzata col metodo associativo su insiemi con 4 linee per set.
- Pentium (tutte le versioni, 1993-1999) – due cache interne
 - Una per i dati ed una per le istruzioni

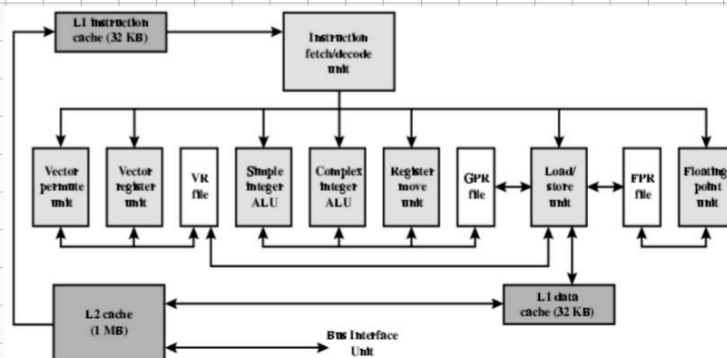
Esempi "storici" di uso cache in CPU Pentium 4 (anni 2000-2008)

- Pentium 4 – Le cache interne (L1), una per dati e una per istruzioni:
 - Dimensione: 8K byte
 - Dimensione linee: 64 byte
 - Metodo usato: associativo su insiemi con quattro linee per set
- Pentium 4 – La cache esterna (L2):
 - alimenta entrambe le cache interne
 - Dimensione: 256K byte
 - Dimensione linee: 128 byte
 - Metodo usato: associativo su insiemi con otto linee per set
 - E' implementata con SRAM (Static RAM)



Esempi "storici" di uso cache in CPU: Power PC (Apple-IBM-Motorola)

- Power PC 601 (1984) – singola cache da 32 Kbyte con metodo associativo su insiemi, con otto linee per set
- 603 – due cache, ciascuna da 8 Kbyte, con metodo associativo su insiemi con due linee per set
- 604 – cache complessiva: 32 Kbyte
- 610 – cache complessiva: 64 Kbyte
- G3 & G4 (usati su computer Apple Power Mac negli anni '90)
 - 64 Kbyte di cache interna
 - metodo associativo su insiemi con otto linee per set
 - 256k, 512k o 1Mbyte di cache esterna
 - metodo associativo su insiemi con due linee per set



Processor	Type	Year of Introduction	L1 cache ^a	L2 cache	L3 cache
IBM 360/85	Mainframe	1968	16 to 32 KB	—	—
PDP-11/70	Minicomputer	1975	1 KB	—	—
VAX 11/780	Minicomputer	1978	16 KB	—	—
IBM 3033	Mainframe	1978	64 KB	—	—
IBM 3090	Mainframe	1985	128 to 256 KB	—	—
Intel 80486	PC	1989	8 KB	—	—
Pentium	PC	1993	8 KB/8 KB	256 to 512 KB	—
PowerPC 601	PC	1993	32 KB	—	—
PowerPC 620	PC	1996	32 KB/32 KB	—	—
PowerPC G4	PC/server	1999	32 KB/32 KB	256 KB to 1 MB	2 MB
IBM S/390 G4	Mainframe	1997	32 KB	256 KB	2 MB
IBM S/390 G6	Mainframe	1999	256 KB	8 MB	—
Pentium 4	PC/server	2000	8 KB/8 KB	256 KB	—
IBM SP	High-end server/supercomputer	2000	64 KB/32 KB	8 MB	—
CRAY MTA ^b	Supercomputer	2000	8 KB	2 MB	—
Itanium	PC/server	2001	16 KB/16 KB	96 KB	4 MB
SGL Origin 2001	High-end server	2001	32 KB/32 KB	4 MB	—

^a Two values separated by a slash refer to instruction and data caches

^b Both caches are instruction only; no data caches

TEMPO DI ACCESSO CACHE - MEMORIA PRIMARIA

$T_s = H T_1 + (1-H)(T_1 + T_2) = T_1 + (1-H)T_2 \rightarrow$ QUESTO È LA FORMULA DEL TEMPO MEDIO T_s PER UN'OPERAZIONE DI LETTURA (O SCRITTURA) QUANDO ESISTE UNA GERARCHIA A DUE LIVELLI (CACHE + MEMORIA PRIMARIA) ^{RAM}

$T_1 \rightarrow$ TEMPO DI ACCESSO ALLA CACHE (LATENZA DI HIT)

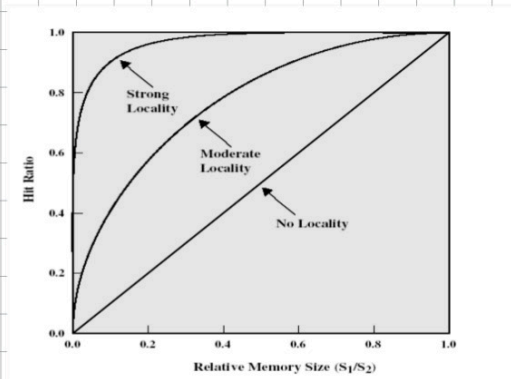
$T_2 \rightarrow$ TEMPO DI ACCESSO ALLA MEMORIA PRIMARIA (RAM)

$H \rightarrow$ HIT RATIO DELLA CACHE (PERCENTUALE DI VOLTE CHE IL DATO CERCATO È TROVATO NELLA MEMORIA CACHE)

• DERIVAZIONE

- $\rightarrow$ CACHE HIT (PROBABILITÀ H): IL DATO SI TROVA IN CACHE, QUINDI COSTA T_1 . CONTRIBUTO MEDIO $\rightarrow H \cdot T_1$
- $\rightarrow$ CACHE MISS (PROBABILITÀ $1-H$):
- PRIMA CONTROLLI CACHE $\rightarrow T_1$
- POI, SE MISS, DEVI ANDARE IN MEMORIA PRIMARIA $\rightarrow T_2$
- TOTALE PER MISS $T_1 + T_2$
- CONTRIBUTO MEDIO $\rightarrow (1-H)(T_1 + T_2)$

SOMMIAMO $\rightarrow T_s = H T_1 + (1-H)(T_1 + T_2) = H T_1 + (1-H)T_1 + (1-H)T_2 = T_1 [H + (1-H)] + (1-H)T_2 = T_1 (1-H)T_2$



TEMPO DI ACCESSO A GERARCHIE A PIÙ LIVELLI

ESTENDIAMO AL CASO "CACHE + RAM" AD UNA GERARCHIA DI N LIVELLI $(L_0, L_1, \dots, L_{N-1})$ CIASCUNO CON ℓ_i TEMPO DI ACCESSO MEDIO AL LIVELLO i , H_i = PROBABILITÀ (HIT RATIO) DI TROVARE IL DATO ENTRO IL LIVELLO i

DEFINIAMO INOLTRE $H_{i|i-1}$ LA HIT RATIO CONDIZIONALE AL LIVELLO i : LA PROBABILITÀ DI TROVARTI A LIVELLO i SAPENDO CHE HAI GIÀ MANCATO IN TUTTI I LIVELLI INFERIORI FINO A $i-1$

• HIT RATIO

- $\rightarrow$ PROBABILITÀ DI TROVARE UNA PAROLA AL LIVELLO i -ESIMO: H_i
- $\rightarrow$ CALCOLO IN TERMINI DI FREQUENZA DI SUCCESSI
- $\rightarrow H_{i+1} > H_i$; $H_{N-1} = 1 \Rightarrow$ I DATI OBTENUTI AL LIVELLO i -ESIMO SONO UN SOTTOINSIEME DEI DATI CONTENUTI AL LIVELLO $(i+1)$ -ESIMO

• HIT RATIO CONDIZIONALE

PROBABILITÀ DI TROVARE UNA PAROLA AL LIVELLO i -ESIMO DATO CHE NON LA SI È TROVATA AL LIVELLO $(i-1)$ -ESIMO $\rightarrow H_{i|i-1}$

TEMPO MEDIO DI ACCESSO ALLA GERARCHIA

• CASO A DUE LIVELLI

- $\rightarrow L_0$ = CACHE: ℓ_0 , HIT RATIO H_0
 - $\rightarrow L_1$ = RAM: ℓ_1 , HIT RATIO $H_1 = 1$
- $T = H_0 \ell_0 + (1-H_0)(\ell_0 + \ell_1) = \ell_0 + (1-H_0)\ell_1$ PRIMA CERCO IN L_0 (SEMPRE ℓ_0), E SOLO SE MISS $(1-H_0)$ PASSO ℓ_1 IN PIÙ

• CASO A 3 LIVELLI

L_0, L_1, L_2 CON $H_2 = 1$. 3 FORME EQUIVALENTI:

- $\rightarrow$ ESPlicita
- $T = H_0 \ell_0 + (H_1 - H_0)(\ell_1 + \ell_0) + (1-H_1)(\ell_2 + \ell_1 + \ell_0)$ $\rightarrow (H_1 - H_0)$ = MISS IN L_0 MA HIT IN $L_1 \rightarrow$ COSTO $\ell_1 + \ell_0$
- $\rightarrow$ HIT CONDIZIONALE
- $T = H_0 \ell_0 + (1-H_0)H_{1|0}(\ell_1 + \ell_0) + (1-H_0)(1-H_{1|0})(\ell_2 + \ell_1 + \ell_0)$ $\rightarrow (1-H_1)$ = MISS FINO A L_1 , QUINDI VAPO IN $L_2 \rightarrow$ COSTO $\ell_2 + \ell_1 + \ell_0$
- $\rightarrow$ FORMA SEMPLIFICATA
- $T = \ell_0 + (1-H_0)\ell_1 + (1-H_1)\ell_2$

• FORMULA GENERALE PER N LIVELLI

INDICHIAMO I LIVELLI DA 0 (PIÙ VELOCE) A $N-1$ (PIÙ LENTO). ALLORA $\rightarrow T = \ell_0 + \sum_{i=1}^{N-1} (1-H_{i-1})\ell_i$ ^{SEMPRE PRESENTE} OGNI TERMINE $(1-H_{i-1})\ell_i$ CONTRIBUISCE SOLO SE SI È MANCATO TUTTI I VALORI FINO A $i-1$